

Kapitel 2. Variabler och datatyper

Innehållsförteckning

[Primitiva datatyper](#)[Deklarera variabler](#)[Tilldelning av variabler](#)[Konstanter](#)

Detta kapitel redogör för de olika primitiva datatyperna som finns i C++, samt hur variabler skapas och tilldelas.

Primitiva datatyper

I C++ finns precis som i de flesta andra programmeringsspråk en mängd olika *datatyper*. Olika datatyper har olika användningsområden och olika aritmetiska operationer fungerar i viss mån olika beroende på datatyp.

Heltal

Den enklaste och vanligaste datatypen är normala *heltal*. Dessa används för diverse beräkningar, iterationer o.s.v. som inte involverar decimaler, endast hela tal. Det finns några olika heltalstyper att välja mellan:

- `int` är grundtypen av heltal. Denna täcker de flesta användningsområden och kan innehålla både positiva och negativa tal. Denna typ är normalt anpassad till datorarkitekturens ordlängd, d.v.s. på en 32-bit system är en `int` även 32 bit stor. Många 64-bit system har dock en 32-bit `int`.
- `long` är normalt en datatyp som är större än en `int`, d.v.s. den kan innehålla större tal. Detta är dock beroende på systemet, men t.ex. flera 64-bit system har en 64-bit stor `long`. En sådan datatyp räcker för att specificera *väldigt* stora tal.
- `short` är kortare än en `int`, ofta endast 16-bit stor. Kan användas då man vet att data som skall finnas i en variabel aldrig är större än att det rymms i ett 16-bit tal.

Alla ovannämnda typer kan innehålla både positiva och negativa tal. Detta halverar dock den teoretiska maximala storleken på de tal datatypen kan representera, eftersom en bit används för att representera tecknet. Ibland kanske man behöver kunna representera mycket stora tal som garanterat är positiva. Då kan man använda en definition `unsigned` som berättar att datatypen som följer `unsigned` (t.ex. `int`) inte skall använda ett förtecken, utan hela datatypen skall användas för att representera positiva tal. Man får då ett maxvärde som är dubbelt så stort.

Talbaser

Man kan i C++ använda olika *talbaser* för att specificera ett tal. I vissa specialtillämpningar är andra talbaser lämpligare än det normala systemet med 10 som grund. Man kan använda *hexadecimala* och *oktala* tal. Den första har basen 16 och den andra har basen 8.

För att specificera ett tal med hexadecimal bas skriver man `0x` eller `0X` (noll-x) före talet. För att använda oktal bas skriver man endast en `0` (noll) före talet. Vi kommer inte i detta kompendium att använda oss av dessa talbaser.

Flyttal

Med *flyttal* i C++ avses tal som har både en heltalsdel och en decimaldel. Flyttal är det som mest liknar normala tal i matematisk mening. Det finns även olika typer av flyttal som har olika precision, precis som heltal ovan. De olika typerna är:

- `float` är den normala versionen på flyttal, precis som `int` för heltal. Denna duger för de flesta tillämnningar. Kan innehålla både positiva och negativa heltal. Denna följer vanligen den använda arkitekturens ordstorlek.
- `double` är en flyttalstyp med dubbel precision jämfört med `float`, därav namnet. Den kan användas där en mängd beräkningar med hög precision behövs.
- `long double` är en ännu längre version av `double` för ökad precision.

Flyttal under C++ lider av vissa speciella problem. Det är omöjligt att representera decimaltal med oändligt antal decimaler m.h.a. några bytes, så många flyttal är avrundningar av de värden de borde innehålla. Om ett stort antal operationer med flyttal utförs kan detta lilla fel ackumuleras och till sist bli så stort att det är signifikat för slutsvaret. Detta problem kan i viss mån avhjälpas genom att använda större flyttalstyper såsom `double` eller `long double`, men på bekostnad av minne.

Tecken

Tecken representeras i C++ av datatypen `char`. En `char` kan innehålla exakt ett tecken. Denna datatyp är egentligen i grund och botten ett heltal som är en byte långt. Tecken representeras m.h.a. koder på 0 till 255, så dessa ryms exakt i en `char`. Man kan dock göra normal matematisk aritmetik med `char`, samt jämföra tecken som man jämför heltal. Nyare implementationer innehåller en alternativ och längre definition på tecken, nämligen `wchar_t`. Denna är ett mycket nytt tillägg, och används inte ännu i så hög grad. Fördelen med denna är att den kan representera ett mycket stort antal olika tecken, så alla språks speciella tecken kan inkodas i samma datatyp. En normal `char` kan ju samtidigt inte innehålla mera än maximalt 256 skilda tecken.

Även `char` kan ges den extra definitionen `unsigned`, varvid dess omfång är 0 till 255, i normala fall är den -127 till 127.

Teckenkonstanter deklarerar genom att innesluta ett tecken mellan två `'`, såsom t.ex. `'a'`, `'9'` eller `'#'`.

Strängar

I C++ finns en datatyp som heter `string` som används för att representera teckensträngar. De som programmerat i C tidigare noterar att detta är ny funktionalitet som uppkommit med de nyaste standarderna av C++. Hur strängar hanteras i C diskuteras i [Appendix A](#). Den nya datatypen är mycket välkommen och gör det lättare speciellt för nybörjare att lära sig C++. En `string` är egentligen mycket mera avancerad än en primitiv datatyp, men vi återkommer till det senare.

En konstant sträng i C++ skrivs innanför citationstecken (`"`). T.ex. är `"Hello world!"` en giltig sträng. Strängen `" "` är en tom sträng. Strängkonstanter kommer att användas extensivt i de flesta av

exemplen i fortsättningen för att skriva ut bl.a. informationstext och resultat. Det finns även några speciella strängkonstanter som används för att utföra speciella funktioner. Dessa är `"\n"` och `"\t"`. Den första används för att skriva ut ett *radbyte*, medan den andra skriver ut ett *tabulatortecken*. Tecknet `\` måste finnas för att kompilatorn skall förstå att det inte är frågan om ett normalt `n` eller `t`, utan att det är specialtecken. Vi kommer dock inte att använda dessa i våra program, men de flesta program använder dem. Se [Kapitel 8](#) för mera information om vad C++ använder för att byta rad.

Booleans

En annan datatyp som introducerats med i och med C++ är datatypen `bool`. Denna är speciell eftersom den har endast två värden som kan användas, nämligen `true` och `false`. Denna datatyp har många tillämpningar, speciellt då olika uttryck skall jämföras, eller då ett visst läge (t.ex. av/på) skall anges. I C emulerade man tidigare datatypen `bool` genom att använda en `int` med värdena 0 eller 1. Många sådana program finns ännu kvar, och endel programmerare föredrar det sättet att representera boolesk funktionalitet. Värdet `true` representeras där av värdet 1 (eller ibland alla värden som inte är 0) och `false` av värdet 0. Program blir dock mycket mera läsbara om man använder `bool` istället för `int`.

[Förutgående](#)

Grundläggande syntax

[Hem](#)

[Nästa](#)

Deklarera variabler